

Sentrais Intelligence Backbone

Technical Design Document

Version: 1.0.0

Date: December 2025

Author: Sentrais Engineering

Executive Summary

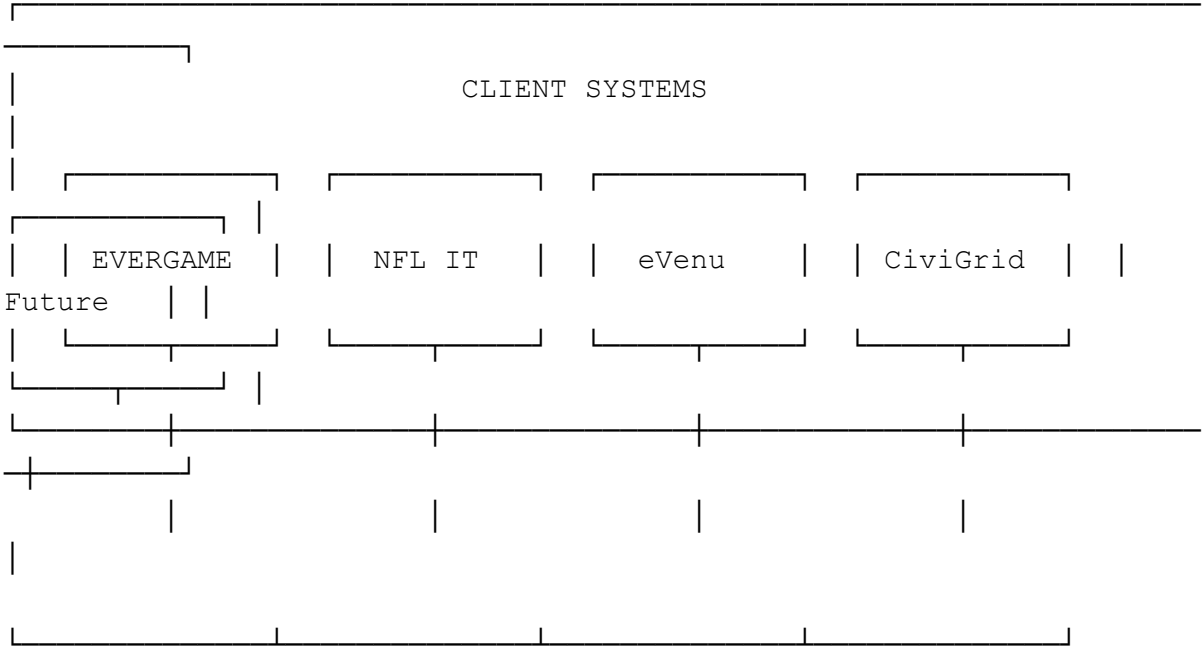
The Sentrais Intelligence Backbone is the central nervous system for operational intelligence across all Sentrais verticals. It captures, stores, analyzes, and learns from operational signals across Sports & Entertainment, Live Events, Aviation, Federal Government, Mega-Events, and Critical Infrastructure.

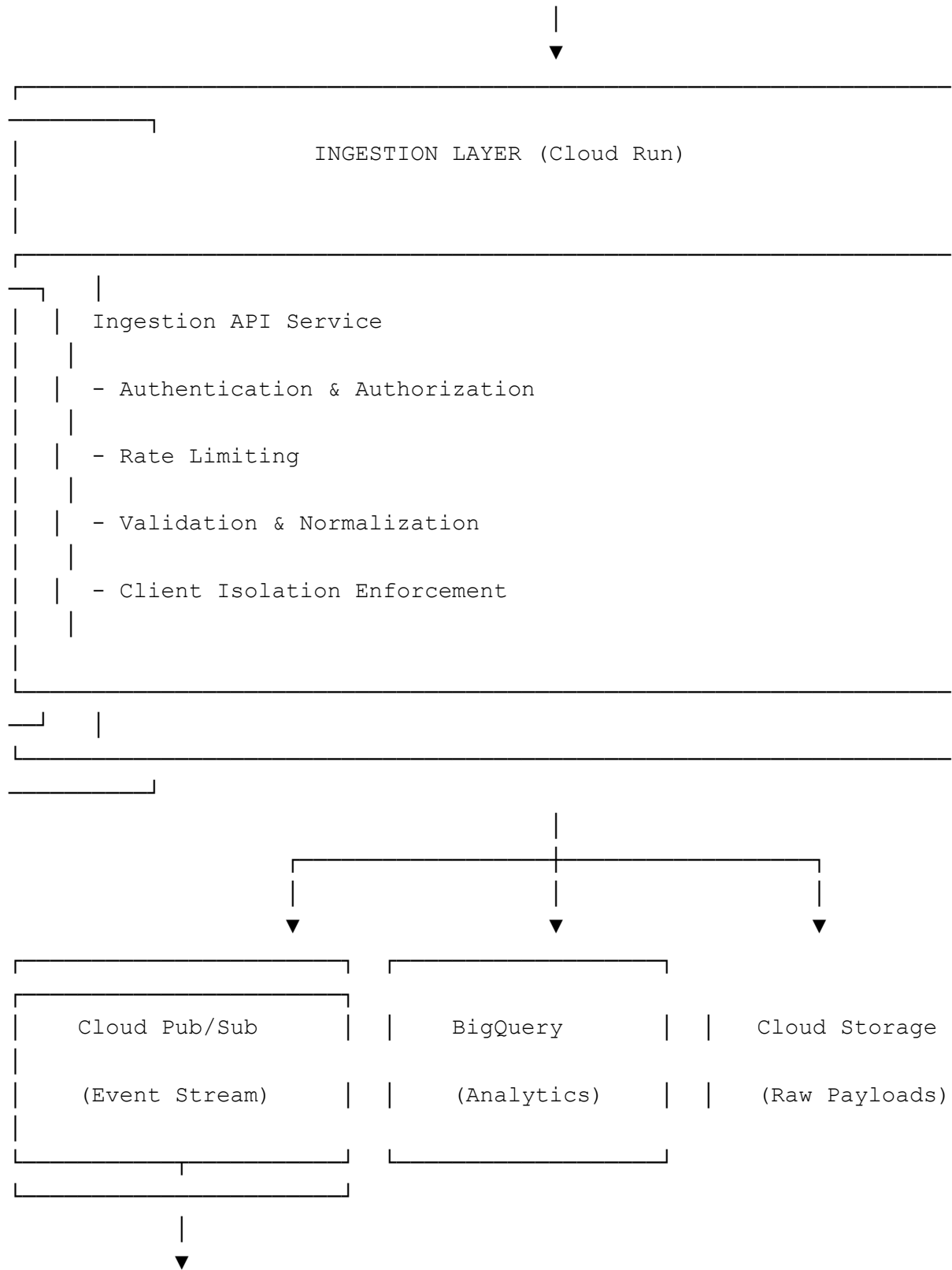
This document describes the technical architecture, data schemas, and API specifications for the backbone system deployed on Google Cloud Platform.

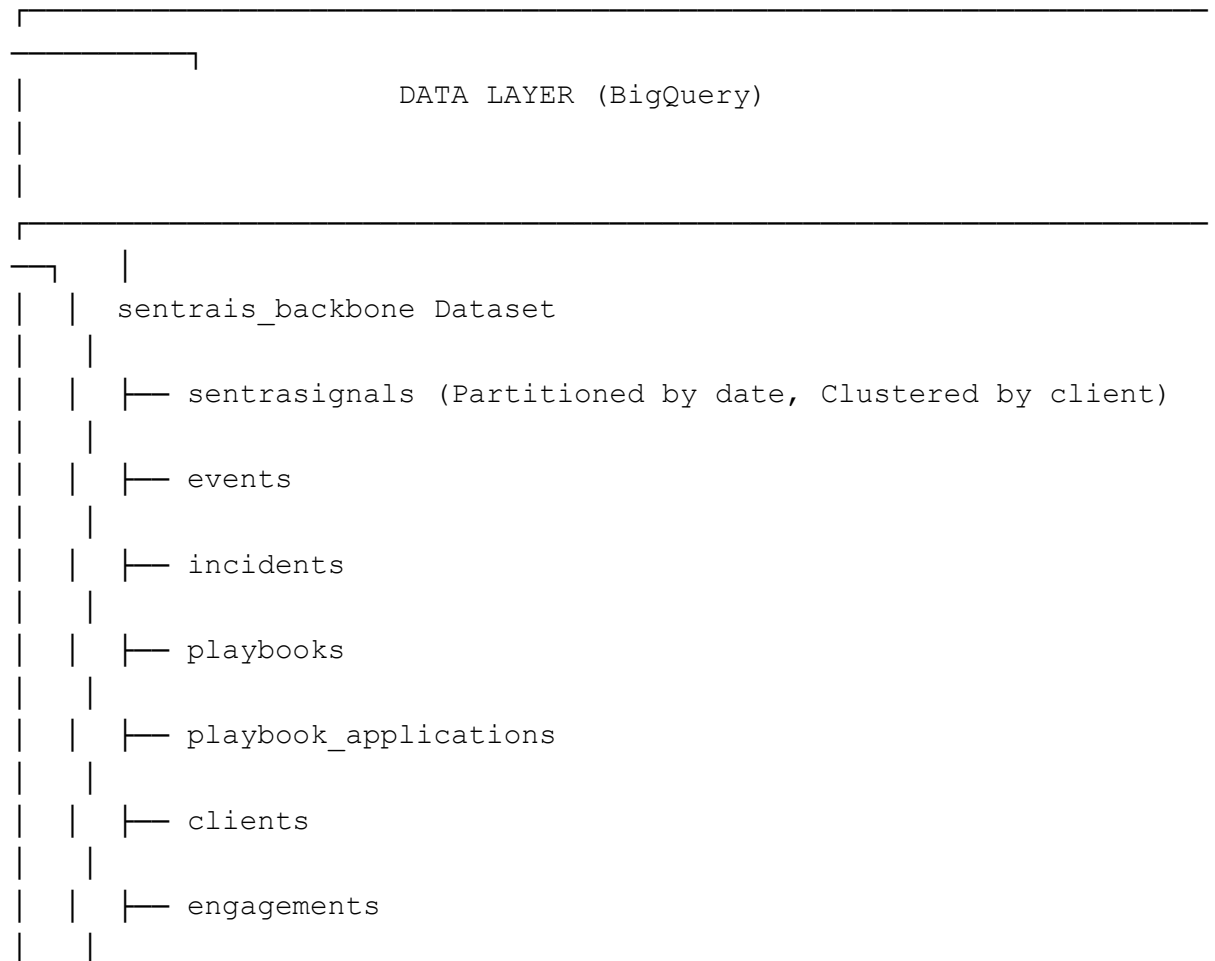
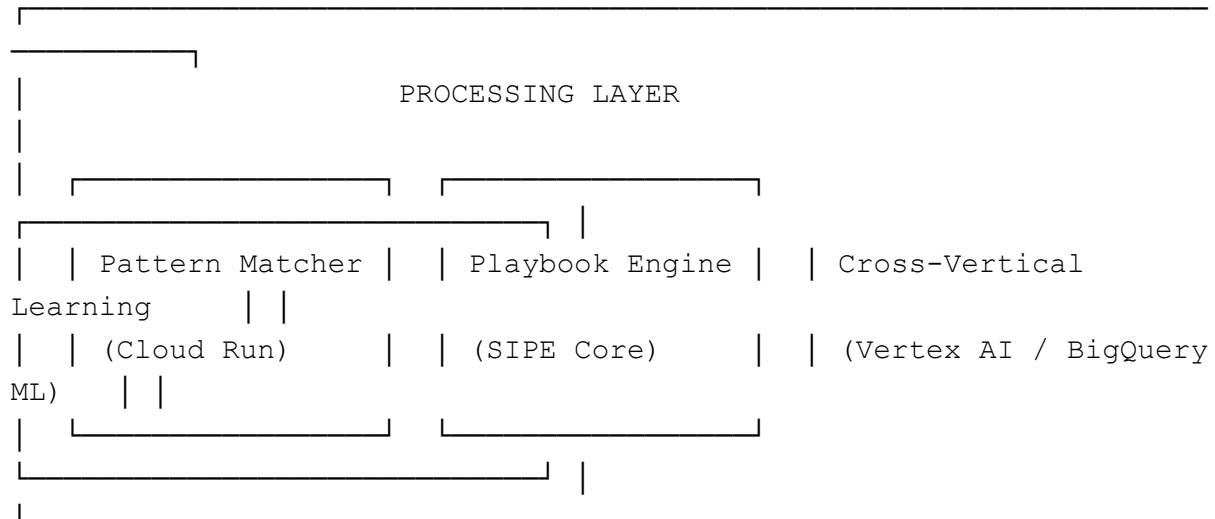
Table of Contents

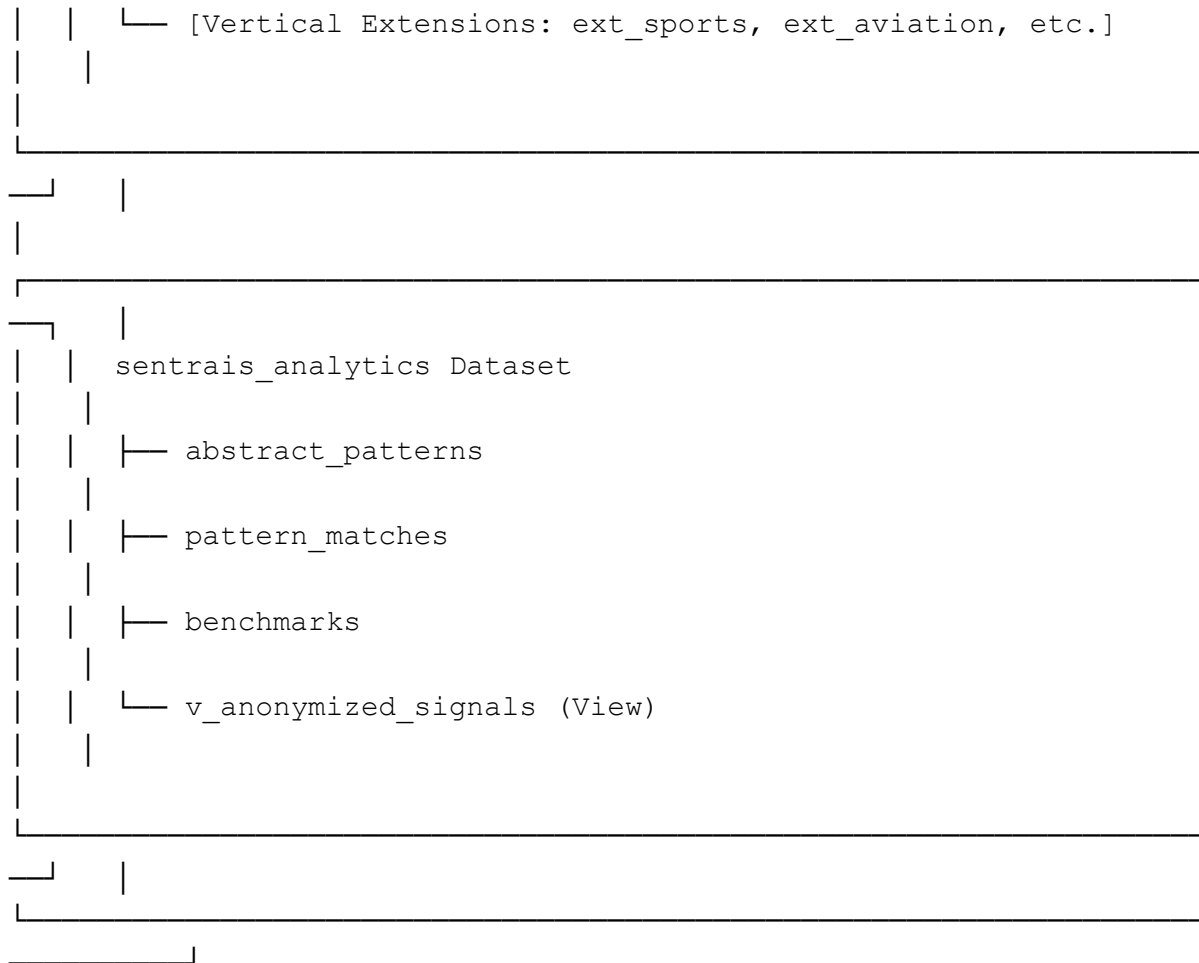
- 1. [Architecture Overview](#)
- 2. [Data Model](#)
- 3. [API Specification](#)
- 4. [Client Separation](#)
- 5. [Cross-Vertical Learning](#)
- 6. [Deployment](#)
- 7. [Security](#)

Architecture Overview







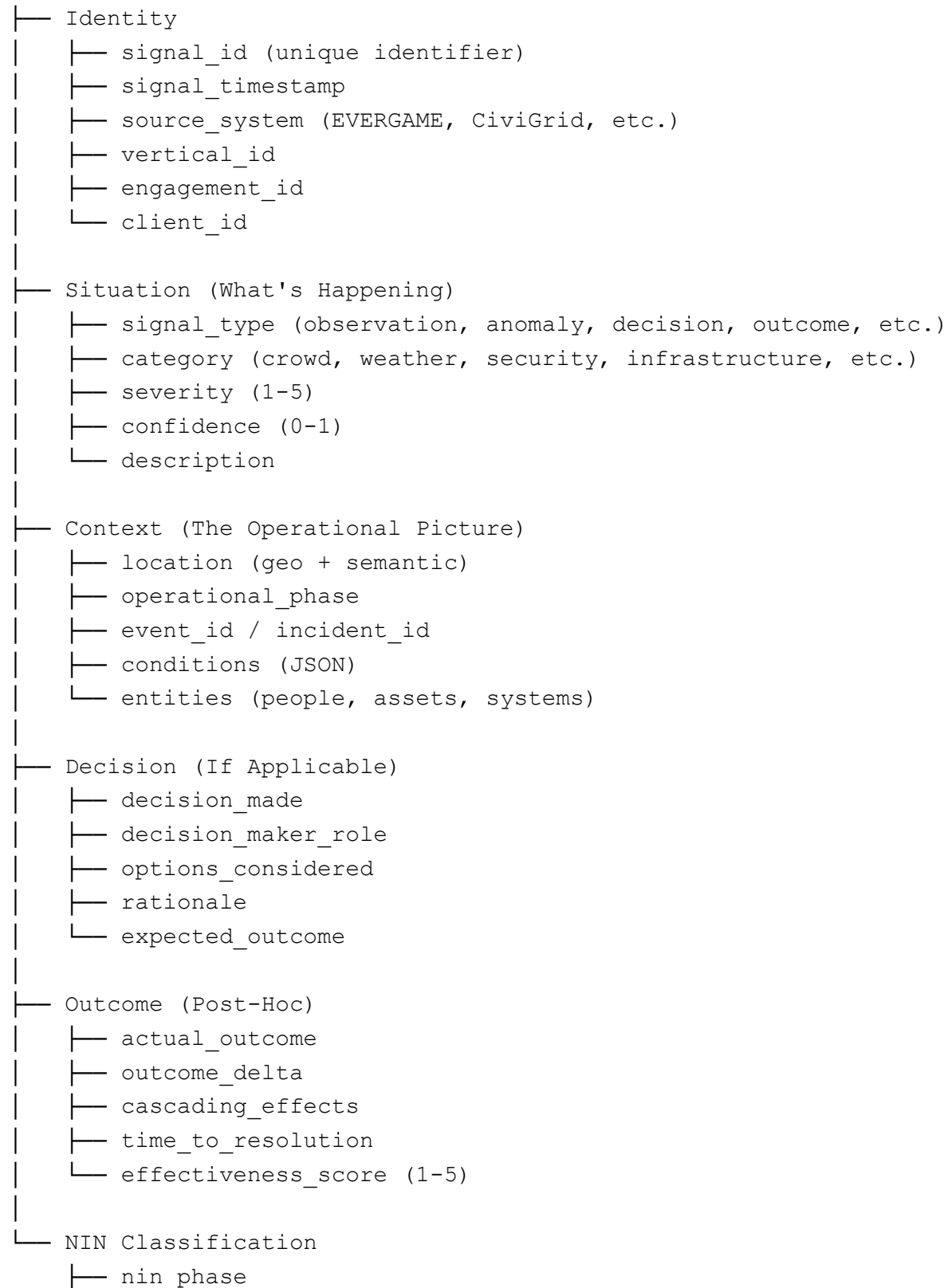


Data Model

The Atomic Unit: SentraSignal

Every piece of operational intelligence is captured as a **SentraSignal**. This is the fundamental building block of the system.

SentraSignal



- |— playbook_id
- |— pattern_tags
- |— cross_vertical_relevance

Vertical Extensions

Each vertical has domain-specific extension tables that capture specialized data:

Vertical	Extension Table	Key Fields
Sports & Entertainment	ext_sports_entertainment	game_state, broadcast_active, vip_attendance
Live Events	ext_live_events	event_genre, production_status, crowd_surfs_incidents
Aviation	ext_aviation_transportation	checkpoint_wait_minutes, ground_stop_active, tsa_threat_level
Federal/Emergency	ext_federal_emergency	fema_region, ics_incident_type, declaration_type
Mega-Events	ext_mega_events	nsse_designation, embassy_notifications, fan_fest_attendance
Critical Infrastructure	ext_critical_infrastructure	ci_sector, operational_status, cyber_status
Venue Operations	ext_venue_operations	gates_open, concession_wait, parking_capacity
Civic/Government	ext_civic_government	public_safety_posture, crowd_sentiment, road_closures

NIN Master Playbook

Playbooks encode institutional operational wisdom:

Playbook

- └ Identity
 - └ playbook_id
 - └ playbook_code (e.g., WEATHER-INGRESS-001)
 - └ playbook_name
- └ Classification
 - └ category_id
 - └ is_universal (cross-vertical?)
 - └ verticals_applicable
- └ Trigger Conditions (JSON)
 - └ When does this playbook activate?
- └ Response Actions (JSON)
 - └ What actions to take, in what order, by whom
- └ Decision Authority
 - └ Who can authorize, who must be notified
- └ Success Metrics
 - └ How do we know it worked?
- └ Evidence Base
 - └ evidence_instances (how many times applied)
 - └ evidence_success_rate
 - └ notable_applications
- └ Cross-Vertical
 - └ derived_from_verticals
 - └ cross_vertical_adaptations

API Specification

Authentication

All API requests require a valid token in the X-Sentraais-Token header:

```
curl -X POST https://api.sentraais.io/v1/signals \
  -H "X-Sentraais-Token: your-token-here" \
  -H "Content-Type: application/json" \
  -d '{"signals": [...]}'
```

Key Endpoints

Method	Endpoint	Description
POST	/v1/signals	Submit signals (batch up to 100)
GET	/v1/signals	Query signals with filters
GET	/v1/signals/{id}	Get specific signal
PATCH	/v1/signals/{id}	Update outcome data
POST	/v1/events	Create event
POST	/v1/incidents	Create incident
POST	/v1/playbooks/match	Find matching playbooks for conditions
POST	/v1/playbooks/{id}/apply	Record playbook application

Example: Submit a Signal

```
curl -X POST https://api.sentraais.io/v1/signals \
-H "X-Sentraais-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "signals": [{
    "signal_type": "threshold_breach",
    "category": "crowd_density",
    "severity": 3,
    "confidence": 0.92,
    "description": "Section 120 density exceeds 0.8 threshold",
    "location": {
      "lat": 33.7553,
      "lng": -84.4006,
      "semantic": "Mercedes-Benz Stadium - Section 120"
    },
    "operational_phase": "active",
    "event_id": "EVT-NFL-2025-ATL-001"
  }]
}'
```

Example: Match Playbooks

```
curl -X POST https://api.sentraais.io/v1/playbooks/match \
-H "X-Sentraais-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "conditions": {
    "weather.precipitation_eta_minutes": 45,
    "crowd.external_crowd_pct": 0.22,
    "operational_phase": "ingress"
  },
  "vertical_id": "sports_entertainment",
  "include_cross_vertical": true
}'
```

```
}'
```

```
---
```

Client Separation

Data Isolation Model

Client data is isolated at multiple layers:

1. **API Layer:** Tokens are scoped to specific clients/engagements
2. **Query Layer:** All queries include `client_id` filter
3. **Storage Layer:** BigQuery row-level security (future)

```
-- All queries automatically filtered
SELECT * FROM sentrasignals
WHERE client_id = 'token_client_id' -- Injected by API
```

Data Sharing Policies

Each client has configurable data sharing policies:

Policy	Description
<code>allow_cross_vertical_learning</code>	Can anonymized data train cross-vertical models?
<code>allow_benchmark_inclusion</code>	Can data be included in benchmarks?

<code>anonymization_delay_days</code>	Days before data can be used (default: 90)
<code>excluded_categories</code>	Categories never shared (e.g., VIP, security)

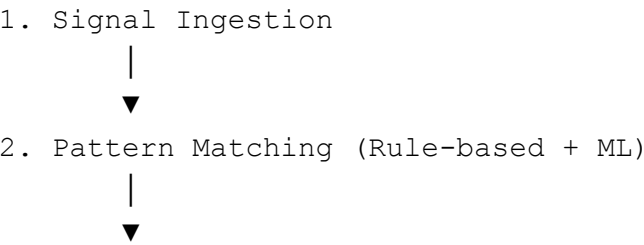
Cross-Vertical Learning

Abstract Patterns

The system identifies abstract patterns that transcend domains:

Pattern	Sports Example	Emergency Example
Large population movement under time pressure	Stadium crowd surge at gates	Hurricane evacuation route surge
High-value asset movement across jurisdictions	VIP motorcade coordination	Critical supply chain coordination
Rapid environment transformation	Halftime entertainment set-up	Emergency shelter conversion

Learning Pipeline



3. Cross-Vertical Correlation



4. Playbook Suggestion



5. Outcome Feedback Loop



6. Model Improvement

Deployment

GCP Resources Required

Resource	Service	Purpose
Ingestion API	Cloud Run	Signal ingestion, API serving
Pattern Matcher	Cloud Run	Real-time pattern detection
Event Streaming	Cloud Pub/Sub	Async processing pipeline
Data Warehouse	BigQuery	Analytics, storage
ML Models	Vertex AI	Pattern recognition, predictions
Secrets	Secret Manager	API keys, credentials
CI/CD	Cloud Build	Automated deployment

Deployment Commands

```
# Deploy BigQuery schemas
bq mk --dataset sentrais_backbone
bq query --use_legacy_sql=false < schemas/01_core_schema.sql
bq query --use_legacy_sql=false < schemas/02_vertical_extensions.sql
bq query --use_legacy_sql=false < schemas/03_playbook_schema.sql
bq query --use_legacy_sql=false < schemas/
04_multitenancy_governance.sql

# Deploy Ingestion API
gcloud run deploy ingestion-api \
  --source services/ingestion_api \
  --region us-central1 \
  --allow-unauthenticated

---
```

Security

Authentication Flow

1. Client obtains API token (out of band)
2. Token stored as SHA-256 hash in api_tokens table
3. Each request validated against token table
4. Token scoped to client_id, engagement_id, permissions
5. Rate limits enforced per token

Data Classification

Level	Description	Examples
public	Can be shared externally	Benchmarks, anonymized patterns
internal	Sentrais use only	Cross-vertical learnings
confidential	Client-specific	Operational signals
restricted	PII, security-sensitive	VIP data, threat intel

File Structure

```
sentrais-backbone/
├── schemas/
│   ├── 01_core_schema.sql           # Core tables (sentrasignals,
│   │                               events, etc.)
│   ├── 02_vertical_extensions.sql   # Vertical-specific extension
│   │                               tables
│   ├── 03_playbook_schema.sql       # NIN Playbook and learning
│   │                               tables
│   └── 04_multitenancy_governance.sql # Client isolation and data
│   │                               governance
│
├── api/
│   └── openapi.yaml                 # Full OpenAPI 3.1
│   │                               specification
│
├── services/
│   └── ingestion_api/
│       ├── main.py                 # FastAPI application
│       └── requirements.txt         # Python dependencies
```

```
|      └─ Dockerfile                # Container definition
|
├─ cloudbuild.yaml                 # CI/CD pipeline
└─ README.md                       # This document

---
```

Next Steps

1. **Deploy BigQuery schemas** to GCP project
2. **Configure Pub/Sub topics** for event streaming
3. **Deploy Ingestion API** to Cloud Run
4. **Create initial API tokens** for EVERGAME integration
5. **Map existing IP** to playbook structure
6. **Integrate EVERGAME** as first client system

Architecting Calm Through Chaos

Sentrais Corporation